

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №4
по дисциплине «Математическая логика и теория формальных
языков»

на тему «Доработка компилятора языка MiLan»
Вариант 38

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Востров А. В. _____

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Описание языка MiLan	4
1.2 Анализ комплексных чисел в программировании	4
1.3 Грамматика языка MiLan	6
1.3.1 Контекстно-свободная грамматика	6
1.3.2 Расширенная грамматика с типами данных	6
1.3.3 Семантические правила типизации	8
1.3.4 Синтаксические диаграммы	9
2 Особенности реализации	13
2.1 Внутреннее представление комплексных чисел	13
2.2 Доработка лексического анализатора	14
2.2.1 Определение новых лексем	14
2.2.2 Названия лексем для сообщений об ошибках	14
2.2.3 Новые ключевые слова	15
2.3 Доработка синтаксического анализатора	16
2.3.1 Типы значений и таблица переменных	16
2.3.2 Приведение и согласование типов	17
2.3.3 Объявление комплексной переменной и согласование ти- пов	18
2.3.4 Разбор выражений со смешением типов	21
2.3.5 Литералы, унарные операции и функции	24
2.3.6 Сравнение значений	26
2.4 Новые инструкции генератора кода	27
2.5 Доработка виртуальной машины	29
3 Результаты работы программы	37
Заключение	43
Список использованных источников	45

Введение

Целью работы является доработка компилятора языка программирования MiLan согласно выбранному варианту.

Вариант 38 Тип данных `complex` и операции над такими числами.

Тип данных `complex` предназначен для хранения комплексных чисел вида $a+bi$, где a – действительная часть, b – мнимая часть, а i – мнимая единица. В рамках работы необходимо встроить поддержку типа `complex` в компилятор языка MiLan и в его виртуальную машину.

Задачи.

1. Проанализировать синтаксическую структуру объявлений и операций над комплексными числами в нескольких языках программирования и выделить требования к типу `complex` в языке MiLan.
2. Расширить форму Бэкуса-Наура языка MiLan.
3. Построить синтаксическую диаграмму расширенной версии языка MiLan.
4. Модифицировать лексический и синтаксический анализаторы компилятора.
5. Реализовать семантические проверки: совместимось типов при присваивании и в арифметических выражениях, а также недопустимые операции над комплексными числами.
6. Реализовать представление комплексных чисел в памяти виртуальной машины и реализовать арифметические операции над ними.

1 Математическое описание

1.1 Описание языка MiLan

Язык MiLan (Mini Language) – это учебный язык программирования.

Программа на языке MiLan представляет собой последовательность операторов заключенных между ключевыми словами **begin** и **end**. Операторы отделяются друг от друга знаком точки с запятой (;). После последнего оператора, после которого следует ключевое слово **end**, точка с запятой не ставится.

Компилятор CMiLan преобразует программы на языке MiLan в последовательность команд виртуальной машины MiLan. Он не чувствителен к регистру символов в именах переменных и ключевых словах.

В синтаксис базовой версии языка входят константы, идентификаторы, арифметические операции над целыми числами, операторы чтения чисел со стандартного ввода и печати чисел на стандартный вывод, а также операторы присваивания, цикла с предусловием и условный. Также программа может содержать комментарии, в том числе многострочные. Вложенные комментарии не допускаются.

В состав компилятора CMiLan входят три основных модуля: лексический анализатор, синтаксический анализатор и генератор команд виртуальной машины. Лексический анализатор читает исходный текст программы и преобразует поток символов в лексемы, попутно пропуская пробелы и комментарии и отслеживая номер текущей строки для сообщений об ошибках. Синтаксический анализатор обрабатывает последовательность лексем методом рекурсивного спуска, проверяет соответствие правилам грамматики и одновременно генерирует команды для виртуальной машины. Синтаксический анализ и генерация кода выполняются в рамках одного прохода по тексту программы.

Виртуальная машина MiLan включает область памяти команд, область памяти данных и стек для промежуточных вычислений. Она выполняет арифметические операции, условные и безусловные переходы, ввод-вывод и в базовой версии работает только с целочисленными значениями.

1.2 Анализ комплексных чисел в программировании

Комплексное число записывается в алгебраической форме как $z = a + bi$, где $a = \operatorname{Re} z$ – действительная часть, $b = \operatorname{Im} z$ – мнимая часть, а i – мнимая единица. Для комплексных чисел определены арифметические операции:

$$(a + bi) + (c + di) = (a + c) + (b + d)i,$$

$$(a + bi) - (c + di) = (a - c) + (b - d)i,$$

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i,$$

$$\frac{a + bi}{c + di} = \frac{ac + bd}{c^2 + d^2} + \frac{bc - ad}{c^2 + d^2} i.$$

Кроме этого, рассматриваются комплексное сопряжение $\bar{z} = a - bi$, модуль $|z| = \sqrt{a^2 + b^2}$ и аргумент $\arg z = \text{atan2}(b, a)$.

В языке C тип комплексных чисел встроен в стандарт, а также используется заголовочный файл `<complex.h>`, в котором определены функции `creal`, `cimag`, `conj`, `cabs`, `carg`. В C++ для этого служит шаблонный класс `complex<T>` из заголовка `<complex>`, перегружающий арифметические операторы и предоставляющий те же функции для работы с комплексными числами.

В Python комплексные числа являются встроенным типом `complex`: литерал записывается как `2 + 3j`, доступны атрибуты `.real`, `.imag`, функция `abs()` и модуль `cmath`.

В языке Fortran поддержка комплексных чисел является встроенной. Комплексный литерал записывается как пара значений в круглых скобках: `(3.0, 4.0)`, где первое – действительная, а второе – мнимая часть. Для работы с комплексными числами определены встроенные функции `REAL()`, `AIMAG()`, `CONJG()` и `ABS()`. Все четыре арифметических оператора (+, -, *, /) работают с типом `COMPLEX`.

Общими чертами рассмотренных языков являются: хранение комплексного числа в виде пары вещественных значений (действительная и мнимая части), автоматическое приведение целочисленных значений к комплексным при смешивании в выражениях, а также наличие стандартного набора функций для выделения частей числа, сопряжения, вычисления модуля и аргумента. Операции порядка (<, >, ≤, ≥) для комплексных чисел, как правило, не определены, поскольку поле комплексных чисел не является упорядоченным; допускается лишь проверка на равенство и неравенство.

На основе проведённого анализа сформулированы следующие требования к реализации типа `complex` в компиляторе MiLan:

1. Язык должен поддерживать оператор объявления комплексной переменной с обязательной инициализацией.
2. Комплексное число должно храниться как пара значений (действительная и мнимая части).
3. Должна поддерживаться запись комплексных и чисто мнимых литералов, а также использование мнимой единицы.
4. Арифметические операции (+, -, *, / и унарный минус) должны работать как над целыми, так и над комплексными значениями; при смешивании типов целое значение должно автоматически приводиться к комплексному.
5. Должны поддерживаться функции выделения действительной и мнимой частей, сопряжения, модуля и аргумента.

6. Для комплексных чисел должны быть разрешены только операции сравнения на равенство и неравенство; операции порядка должны диагностироваться как ошибка.
7. Присваивание комплексного значения целой переменной должно диагностироваться как ошибка.

1.3 Грамматика языка MiLan

1.3.1 Контекстно-свободная грамматика

Грамматика языка MiLan относится к классу контекстно-свободных грамматик (тип 2 по иерархии Хомского), то есть для нее справедливо следующее:

- все правила имеют вид $A \rightarrow \alpha$, где A принадлежит множеству нетерминалов языка, а α является любой последовательностью из терминальных и нетерминальных символов;
- правые части продукций могут содержать любые комбинации нетерминалов и терминалов и не зависят от контекста.

Синтаксический анализатор CMiLan реализован методом рекурсивного спуска и является LL(1)-парсером: каждому нетерминалу грамматики соответствует отдельная процедура, а выбор альтернативы осуществляется на основе одного текущего токена. Стек вызовов процедур играет роль магазина автомата с магазинной памятью, используемого при LL(1)-разборе. Разбор и генерация кода виртуальной машины выполняются за один проход по входному тексту программы.

1.3.2 Расширенная грамматика с типами данных

Грамматика языка в форме Бэкуса-Наура приведена далее.

```

<program>      ::= 'begin' <statementList> 'end'
<statementList> ::= <statement> { ';' <statement> } | ε
<statement>    ::= <ident> := <expression>
                | <ident> ':= ' <expression>
                | 'complex' <ident> ':= ' <expression>
                | 'if' <relation> 'then' <statementList> ['else'
                  <statementList>] 'fi'
                | 'while' <relation> 'do' <statementList> 'od'
                | 'write' '(' <expression> ')'
<expression>   ::= <term> { <addop> <term> }
<term>         ::= <factor> { <multop> <factor> }
```

$$\begin{aligned}
\langle factor \rangle &::= \langle ident \rangle \mid '(\langle expression \rangle)'\mid 'read' \\
&\mid '-' \langle factor \rangle \mid '+' \langle factor \rangle \\
&\mid \langle number \rangle ['i'] \mid 'i' \\
&\mid 'real' '(\langle expression \rangle)'\mid \\
&\mid 'imag' '(\langle expression \rangle)'\mid \\
&\mid 'conj' '(\langle expression \rangle)'\mid \\
&\mid 'abs' '(\langle expression \rangle)'\mid \\
&\mid 'arg' '(\langle expression \rangle)'\mid \\
\langle relation \rangle &::= \langle expression \rangle \langle cmp \rangle \langle expression \rangle \\
\langle addop \rangle &::= '+' \mid '-' \\
\langle multop \rangle &::= '*' \mid '/' \\
\langle cmp \rangle &::= '=' \mid '!=' \mid '<' \mid '<=' \mid '>=' \mid '>' \\
\langle ident \rangle &::= \langle letter \rangle \{ \langle letter \rangle \mid \langle digit \rangle \} \\
\langle number \rangle &::= \langle digit \rangle + \\
\langle letter \rangle &::= a \mid b \mid \dots \mid Z \\
\langle digit \rangle &::= 0 \mid 1 \mid \dots \mid 9
\end{aligned}$$

Добавленные расширения.

1. Добавлен оператор `complex <ident> := <expression>` – явное объявление переменных комплексного типа с обязательной инициализацией. В качестве инициализатора комплексной переменной допускается произвольное выражение; если его значение целое, оно автоматически приводится к комплексному.
2. В качестве фактора допускаются записи `<number> i` (чисто мнимое число) и `i` (мнимая единица). Комплексное число вида $a + bi$ при этом не является отдельным литералом: оно получается как обычное выражение – сумма целого фактора a и мнимого фактора bi со смешением типов.
3. В качестве фактора допускаются унарный минус – `<factor>` и унарный плюс `+ <factor>`, работающие как над целыми, так и над комплексными значениями.
4. Добавлены функции для работы с комплексными числами:
 - `real(z)` – выделение действительной части,
 - `imag(z)` – выделение мнимой части,
 - `conj(z)` – комплексное сопряжение $\bar{z}$,
 - `abs(z)` – модуль комплексного числа,
 - `arg(z)` – аргумент комплексного числа.

1.3.3 Семантические правила типизации

Синтаксически допустимые программы образуют контекстно-свободный язык, однако условия типовой корректности — допустимость конкретного оператора сравнения, совместимость типов при присваивании — зависят от контекста объявлений переменных и не могут быть выражены в БНФ. Они относятся к контекстно-зависимым ограничениям и проверяются компилятором отдельно, в ходе семантического анализа.

Тип каждого выражения — `int` или `complex` — определяется по следующим правилам.

Типы факторов.

- $\langle number \rangle$ имеет тип `int`.
- $\langle number \rangle$ 'i' и 'i' имеют тип `complex`.
- $\langle ident \rangle$ наследует тип из своего объявления.
- $\langle ' e ' \rangle$ имеет тип выражения e .
- $real(e)$, $imag(e)$, $conj(e)$, $abs(e)$, $arg(e)$ имеют тип `complex`; если e имеет тип `int`, он автоматически приводится к `complex`.
- Унарный $-f$ и $+f$ сохраняют тип f .

Типы составных выражений. Для бинарной операции $e_1 \text{ op } e_2$ (где $op \in \{+, -, *, /\}$):

- оба операнда `int` $\Rightarrow$ результат `int`;
- хотя бы один операнд `complex` $\Rightarrow$ результат `complex`.

Правила присваивания.

- `complex` $x := e$ — допустимо при любом типе e ; переменная x не должна быть уже объявлена; тип `int` автоматически приводится к `complex`.
- $x := e$, где x объявлена как `complex` — допустимо при любом типе e ; тип `int` автоматически приводится.
- $x := e$, где x не объявлена или объявлена как `int` — допустимо только если тип e равен `int`; присваивание `complex`-значения целой переменной является ошибкой.

Правила отношений. Для $\langle relation \rangle = e_1 \langle cmp \rangle e_2$:

- если оба выражения имеют тип `int` — допустимы все шесть операторов `=`, `!=`, `<`, `<=`, `>`, `>=`;
- если хотя бы одно выражение имеет тип `complex` — допустимы только `=` и `!=`; операторы `<`, `<=`, `>`, `>=` диагностируются как ошибка типизации.

1.3.4 Синтаксические диаграммы

На рис. 1-2 приведена расширенная синтаксическая диаграмма.

- `y_stop` – `STOP`;
- `y_decl` – `STORE addr`;
- `y_cdecl` – если выражение `VT_INT` – `PUSH 1000, MULT, PUSH 0`; затем `CSTORE addr`.
- `y_fi` –
- `y_od` – `JUMP condAddr`;
- `y_write` – `PRINT / CPRINT`;
- `y_load` – `LOAD addr / CLOAD addr`;
- `y_push` – `PUSH value`;
- `y_read` – `INPUT`;
- `y_invert` – `INVERT / CINVERT`;
- `y_cpushi` – `CPUSHI value`;
- `y_cpushi1` – `CPUSHI 1`;
- `y_creal` – если аргумент `VT_INT` : `PUSH 1000, MULT, PUSH 0`, затем `CREAL`;
- `y_imag` – если аргумент `VT_INT` : `PUSH 1000, MULT, PUSH 0`, затем `CIMAG`;
- `y_conj` – если аргумент `VT_INT` : `PUSH 1000, MULT, PUSH 0`, затем `CCONJ`;
- `y_abs` – если аргумент `VT_INT` : `PUSH 1000, MULT, PUSH 0`, затем `CABS`;

- `y_arg` – если аргумент `VT_INT` : `PUSH 1000, MULT, PUSH 0`, затем `CARG`;
- `y_add` – `ADD / CADD`;
- `y_sub` – `SUB / CSUB`;
- `y_mult` – `MULT / CMULT`;
- `y_div` – `DIV / CDIV`;
- `y_cmp` – `COMPARE op / CCOMPARE op`.

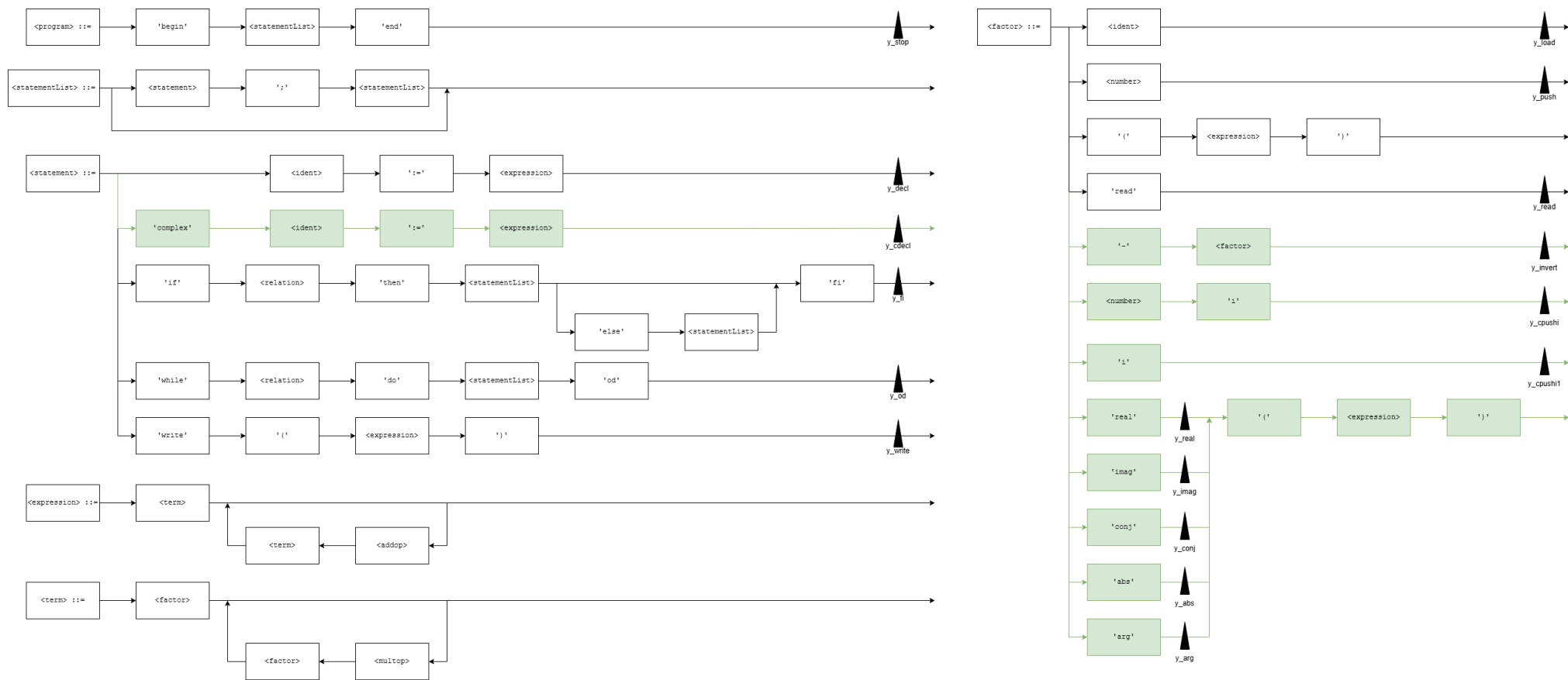


Рис. 1: Синтаксическая диаграмма SMilan

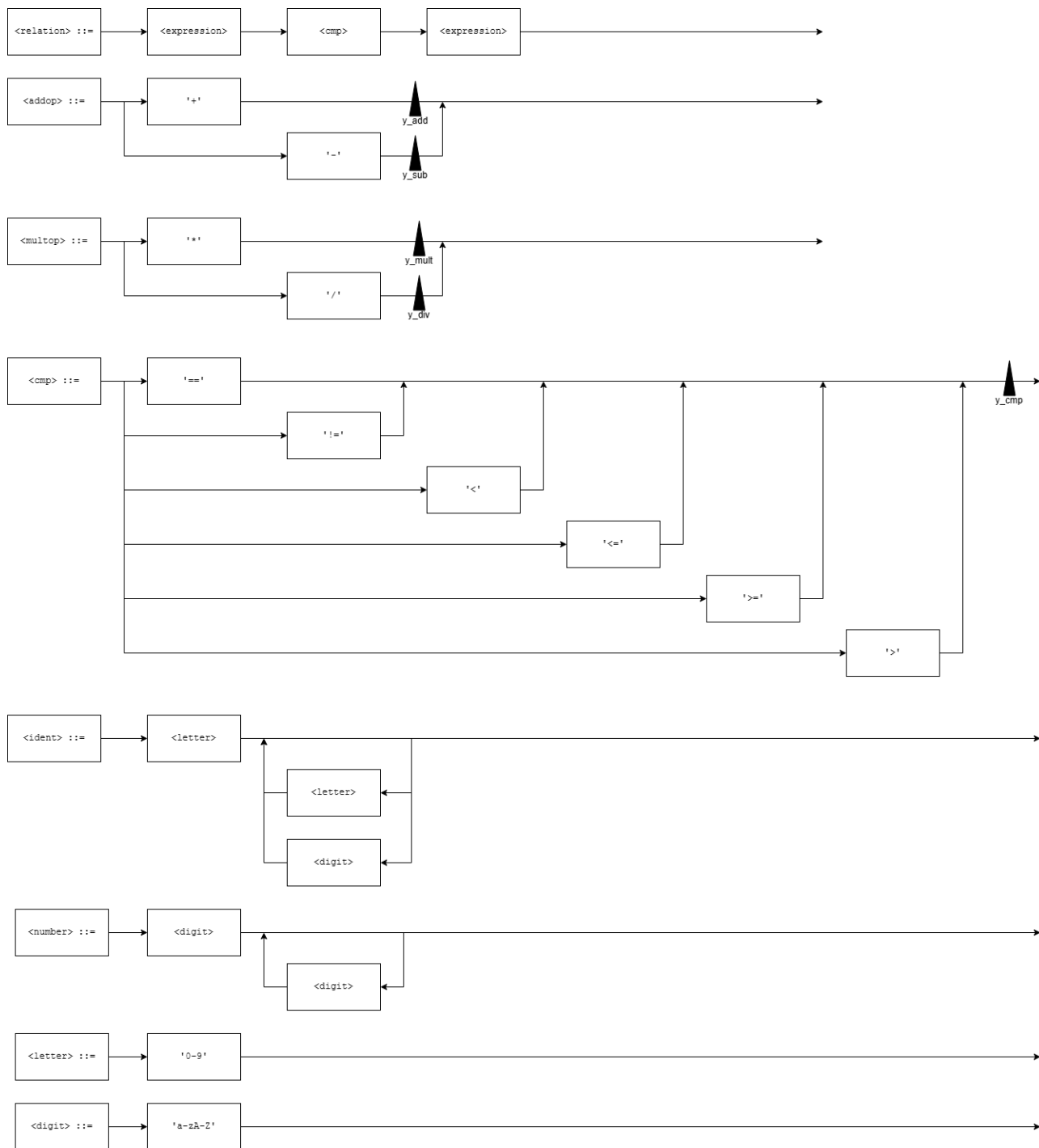


Рис. 2: Синтаксическая диаграмма CMilan

2 Особенности реализации

Доработка затрагивает все модули проекта: лексический анализатор (`scanner.h`, `scanner.cpp`), синтаксический анализатор (`parser.h`, `parser.cpp`), генератор машинных команд (`codegen.h`, `codegen.cpp`) и виртуальную машину (`vm.h`, `vm.c`, `main.c`).

В листингах доработанных методов (тех, что существовали в базовой версии компилятора):

- жёлтым фоном выделены строки, изменённые по сравнению с оригинальной реализацией;
- зелёным фоном выделены строки, добавленные как новые.

2.1 Внутреннее представление комплексных чисел

Представление комплексного числа реализуется парой целых значений. Каждая часть: действительная и мнимая – хранится как целое число, равное $\text{round}(x \cdot \text{SCALE})$, где масштаб $\text{SCALE} = 1000$, что обеспечивает точность три знака после запятой. Масштаб задаётся одинаково в компиляторе и в виртуальной машине: в компиляторе – константой `VM_SCALE` в `parser.cpp`, в машине – макросом `VM_SCALE` в `vm.h`.

На стеке виртуальной машины комплексное число занимает две ячейки: ниже лежит действительная часть, выше – мнимая. В памяти комплексная переменная занимает две соседних ячейки: по адресу переменной – действительная часть, по следующему адресу – мнимая.

Листинг 1 Масштаб с фиксированной точкой

```
1 // parser.cpp
2 static const int VM_SCALE = 1000;
3
4 // vm.h
5 #define VM_SCALE 1000
```

В `parser.cpp` объявлены адреса трёх служебных временных ячеек памяти в верхней части адресного пространства. Они используются для временного сохранения операнда при согласовании типов, когда требуется переупорядочить значения на стеке (листинг 2).

Листинг 2 Служебные временные ячейки памяти

```
1 // parser.cpp
2 static const int TMP_SLOT_0 = 65530;
3 static const int TMP_SLOT_1 = 65531;
4 static const int TMP_SLOT_2 = 65532;
```

2.2 Доработка лексического анализатора

2.2.1 Определение новых лексем

В лексическом анализаторе перечисление Token (scanner.h) дополнено лексемами для распознавания нового ключевого слова `complex`, функций над комплексными числами и мнимой единицы. Добавлены лексемы T_COMPLEX, T_REAL, T_IMAG, T_CONJ, T_ABS, T_ARG и T_IMAG_UNIT. Изменения представлены в листинге 3, строки 25-31.

Листинг 3 Элементы Token

```
1 // scanner.h
2 enum Token {
3     T_EOF,          // Конец текстового потока
4     T_ILLEGAL,      // Признак недопустимого символа
5     T_IDENTIFIER,   // Идентификатор
6     T_NUMBER,       // Целочисленный литерал
7     T_BEGIN,        // Ключевое слово "begin"
8     T_END,          // Ключевое слово "end"
9     T_IF,           // Ключевое слово "if"
10    T_THEN,          // Ключевое слово "then"
11    T_ELSE,          // Ключевое слово "else"
12    T_FI,            // Ключевое слово "fi"
13    T_WHILE,         // Ключевое слово "while"
14    T_DO,            // Ключевое слово "do"
15    T_OD,            // Ключевое слово "od"
16    T_WRITE,         // Ключевое слово "write"
17    T_READ,          // Ключевое слово "read"
18    T_ASSIGN,        // Оператор "!="
19    T_ADDOP,         // Сводная лексема для "+" и "-"
20    T_MULOP,         // Сводная лексема для "*" и "/"
21    T_CMP,           // Сводная лексема для операторов отношения
22    T_LPAREN,        // Открывающая скобка
23    T_RPAREN,        // Закрывающая скобка
24    T_SEMICOLON,     // ";"
25    T_COMPLEX,       // Ключевое слово "complex"
26    T_REAL,          // функция real()
27    T_IMAG,          // функция imag()
28    T_CONJ,          // функция conj()
29    T_ABS,           // функция abs()
30    T_ARG,           // функция arg()
31    T_IMAG_UNIT      // мнимая единица "i"
32 };
```

2.2.2 Названия лексем для сообщений об ошибках

Для корректного вывода сообщений об ошибках в массив `tokenNames_` (scanner.cpp) добавлены строковые представления новых лексем. Изменения приведены в листинге 4, строки 25-31.

Листинг 4 Добавленные элементы массива tokenNames_

```

1 // scanner.cpp
2 static const char * tokenNames_[] = {
3     "end of file",
4     "illegal token",
5     "identifier",
6     "number",
7     "'BEGIN'",
8     "'END'",
9     "'IF'",
10    "'THEN'",
11    "'ELSE'",
12    "'FI'",
13    "'WHILE'",
14    "'DO'",
15    "'OD'",
16    "'WRITE'",
17    "'READ'",
18    "':=' ",
19    "'+' or '-' ",
20    "'*' or '/' ",
21    "comparison operator",
22    "'(' ",
23    "')' ",
24    "';' ",
25    "'COMPLEX'",
26    "'REAL'",
27    "'IMAG'",
28    "'CONJ'",
29    "'ABS'",
30    "'ARG'",
31    "'i'"
32 };

```

2.2.3 Новые ключевые слова

Чтобы лексический анализатор распознавал слова `complex`, `real`, `imag`, `conj`, `abs`, `arg` и `i` как ключевые, они внесены в словарь `keywords_` в конструкторе класса `Scanner` (`scanner.h`). Изменения представлены в листинге 5, строки 16-22.

Листинг 5 Добавленные ключевые слова в конструкторе Scanner

```

1 // scanner.h
2 explicit Scanner(const string& fileName, istream& input)
3     : fileName_(fileName), lineNumber_(1), input_(input)
4 {
5     keywords_["begin"]    = T_BEGIN;
6     keywords_["end"]      = T_END;
7     keywords_["if"]       = T_IF;
8     keywords_["then"]     = T_THEN;

```

```

9      keywords_["else"]      = T_ELSE;
10     keywords_["fi"]       = T_FI;
11     keywords_["while"]    = T_WHILE;
12     keywords_["do"]       = T_DO;
13     keywords_["od"]       = T_OD;
14     keywords_["write"]    = T_WRITE;
15     keywords_["read"]     = T_READ;
16     keywords_["complex"]  = T_COMPLEX;
17     keywords_["real"]     = T_REAL;
18     keywords_["imag"]     = T_IMAG;
19     keywords_["conj"]     = T_CONJ;
20     keywords_["abs"]      = T_ABS;
21     keywords_["arg"]      = T_ARG;
22     keywords_["i"]        = T_IMAG_UNIT;
23
24     nextChar();
25 }

```

2.3 Доработка синтаксического анализатора

2.3.1 Типы значений и таблица переменных

Для синтаксического анализатора была введена системы типов с двумя типами значений. В класс `Parser` (`parser.h`) добавлено перечисление `VarType` с тремя значениями: `VT_INT` (целое), `VT_COMPLEX` (комплексное) и `VT_ERROR` (признак ошибки типизации). Изменения представлены в листинге 6.

Листинг 6 Перечисление типов значений

```

1 // parser.h
2 enum VarType {
3     VT_INT,          // обычное целое
4     VT_COMPLEX,      // комплексное число (re, im)
5     VT_ERROR         // ошибка типизации
6 };

```

Для хранения сведений о переменной добавлена структура `VarInfo`, содержащая адрес переменной в памяти и её тип (листинг 7).

Листинг 7 Структура `VarInfo`

```

1 // parser.h
2 struct VarInfo {
3     int address;
4     VarType type;
5     VarInfo() : address(0), type(VT_INT) {}
6     VarInfo(int a, VarType t) : address(a), type(t) {}
7 };

```


Прежняя таблица переменных, хранившая только адреса, заменена таблицей вида `map<string, VarInfo>`, где каждой переменной сопоставлены и адрес, и тип (листинг 8).

Листинг 8 Таблица переменных

```
1 // parser.h
2 typedef map<string, VarInfo> VarTable;
3 VarTable variables_;
```

Также изменены методы доступа к таблице. Метод `getVariable` ищет переменную по имени и заполняет структуру `VarInfo`, возвращая признак успеха (листинг 9).

Вход: имя переменной `name` типа `const string&`; ссылка `out` типа `VarInfo&` — структура, куда будет записан результат при успешном поиске.

Выход: `true`, если переменная найдена в таблице (при этом `out` заполняется адресом и типом); `false`, если переменная не объявлена.

Листинг 9 Метод `getVariable`

```
1 bool Parser::getVariable(const string& name, VarInfo& out)
2 {
3     VarTable::iterator it = variables_.find(name);
4     if(it == variables_.end()) return false;
5     out = it->second;
6     return true;
7 }
```

Метод `addVariable` регистрирует новую переменную заданного типа: для комплексной выделяются два слова памяти, для целой — одно (листинг 10).

Вход: имя переменной `name` типа `const string&`; тип переменной `type` типа `VarType`.

Выход: адрес `address` типа `int` — базовый адрес выделенной ячейки (или первой из двух ячеек для комплексной переменной).

Листинг 10 Метод `addVariable`

```
1 int Parser::addVariable(const string& name, VarType type)
2 {
3     int address = lastVar_;
4     variables_[name] = VarInfo(address, type);
5     if(type == VT_COMPLEX) lastVar_ += 2;
6     else lastVar_ += 1;
7     return address;
8 }
```

2.3.2 Приведение и согласование типов

Для автоматического приведения целого значения к комплексному добавлен метод `promoteIntToComplex`.

Вход: неявный — на вершине стека виртуальной машины должно находиться одно слово с целым значением n .

Выход: стек виртуальной машины модифицирован: вместо одного слова n на вершине лежат два слова, представляющих комплексное число $(n \cdot SCALE, 0)$ — действительная часть под мнимой (равной нулю).

Он преобразует находящееся на вершине стека целое слово n в пару $(n \cdot SCALE, 0)$: значение умножается на масштаб (становясь действительной частью комплексного числа), после чего сверху помещается нулевая мнимая часть (листинг 11).

Листинг 11 Приведение целого к комплексному

```
1 // parser.cpp
2 void Parser::promoteIntToComplex()
3 {
4     codegen_ ->emit(PUSH, VM_SCALE);
5     codegen_ ->emit(MULT);    // n*SCALE = действительная часть
6     codegen_ ->emit(PUSH, 0); // 0 = мнимая часть, сверху
7 }
```

Метод `unifyTypes` определяет результирующий тип операции по типам операндов: если хотя бы один операнд комплексный, результат комплексный; при наличии ошибки типизации результат — ошибка.

Вход: тип левого операнда `l` и тип правого операнда `r`, оба типа `VarType`.

Выход: результирующий тип `VarType`: `VT_ERROR`, если хотя бы один из входов равен `VT_ERROR`; `VT_COMPLEX`, если хотя бы один из входов равен `VT_COMPLEX`; `VT_INT` в остальных случаях.

Код приведен в листинге 12.

Листинг 12 Согласование типов операндов

```
1 // parser.cpp
2 VarType Parser::unifyTypes(VarType l, VarType r)
3 {
4     if(l == VT_ERROR || r == VT_ERROR) return VT_ERROR;
5     if(l == VT_COMPLEX || r == VT_COMPLEX) return VT_COMPLEX;
6     return VT_INT;
7 }
```

2.3.3 Объявление комплексной переменной и согласование типов

Для обработки объявления вида `complex <ident> := <expression>` в метод `statement` добавлена ветвь, срабатывающая при встрече ключевого слова `complex` и передающая управление новому методу `complexDeclaration` (листинг 13, строки 4-7).

Также метод `statement` был доработан так, чтобы он учитывал типы. При присваивании уже объявленной переменной (`<ident> := <expression>`) определяется тип переменной и тип значения выражения. Если переменная

ранее не была объявлена, она считается целой. Затем выполняется согласование: присваивание комплексного значения целой переменной запрещается и диагностируется как ошибка, а при присваивании целого значения комплексной переменной целое автоматически приводится к комплексному. В зависимости от типа переменной значение записывается командой `CSTORE` или `STORE`. Изменения затронули строки 16-36.

Оператор печати `write` также изменён: после разбора выражения по его типу выбирается инструкция печати — `CPRINT` для комплексного значения и `PRINT` для целого. Изменения в строках 66-74.

Структура операторов `if` и `while` осталась прежней.

Полный листинг метода с выделенными изменениями приведён в листинге 13.

Листинг 13 Ветвь объявления комплексной переменной в `statement`

```
1 // parser.cpp
2 void Parser::statement()
3 {
4     if(match(T_COMPLEX)) {
5         complexDeclaration();
6         return;
7     }
8
9     if(see(T_IDENTIFIER)) {
10         string name = scanner_ ->getStringValue();
11         next();
12         mustBe(T_ASSIGN);
13
14         VarInfo varInfo;
15         bool exists = getVariable(name, varInfo);
16         if(!exists) {
17             // если переменная не объявлена, считаем её целой
18             int addr = addVariable(name, VT_INT);
19             varInfo = VarInfo(addr, VT_INT);
20         }
21
22         VarType exprType = expression();
23         if(exprType == VT_ERROR) return;
24
25         if(varInfo.type == VT_INT && exprType == VT_COMPLEX) {
26             reportError("cannot assign complex value to int variable '"
27                 + name + "'.");
28             return;
29         }
30         if(varInfo.type == VT_COMPLEX && exprType == VT_INT) {
31             promoteIntToComplex();
32         }
33         if(varInfo.type == VT_COMPLEX) {
34             codegen_ ->emit(CSTORE, varInfo.address);
35         }
36     }
37 }
```

```

36     else {
37         codegen_ -> emit(STORE, varInfo.address);
38     }
39 }
40 else if (match(T_IF)) {
41     relation();
42     int jumpNoAddress = codegen_ -> reserve();
43     mustBe(T_THEN);
44     statementList();
45     if (match(T_ELSE)) {
46         int jumpAddress = codegen_ -> reserve();
47         codegen_ -> emitAt(jumpNoAddress, JUMP_NO, codegen_ ->
            getCurrentAddress());
48         statementList();
49         codegen_ -> emitAt(jumpAddress, JUMP, codegen_ ->
            getCurrentAddress());
50     }
51     else {
52         codegen_ -> emitAt(jumpNoAddress, JUMP_NO, codegen_ ->
            getCurrentAddress());
53     }
54     mustBe(T_FI);
55 }
56 else if (match(T_WHILE)) {
57     int conditionAddress = codegen_ -> getCurrentAddress();
58     relation();
59     int jumpNoAddress = codegen_ -> reserve();
60     mustBe(T_DO);
61     statementList();
62     mustBe(T_OD);
63     codegen_ -> emit(JUMP, conditionAddress);
64     codegen_ -> emitAt(jumpNoAddress, JUMP_NO, codegen_ ->
        getCurrentAddress());
65 }
66 else if (match(T_WRITE)) {
67     mustBe(T_LPAREN);
68     VarType t = expression();
69     mustBe(T_RPAREN);
70     if (t == VT_ERROR) return;
71     if (t == VT_COMPLEX) {
72         codegen_ -> emit(CPRINT);
73     }
74     else {
75         codegen_ -> emit(PRINT);
76     }
77 }
78 else {
79     reportError("statement expected.");
80 }
81 }

```

Метод complexDeclaration

Вход: неявный — текущая позиция в потоке лексем стоит сразу после уже потреблённого токена `complex`; в потоке ожидается последовательность `<ident> ':' '=' <expression>`.

Выход: нет возвращаемого значения; новая комплексная переменная зарегистрирована в таблице, в буфер кодогенератора добавлены инструкции вычисления инициализирующего выражения и сохранения результата командой `CSTORE`. При ошибке устанавливается флаг `error_`.

Метод считывает имя переменной, проверяет, что она ещё не объявлена, регистрирует её как комплексную, после обязательной лексемы «:=» разбирает выражение-инициализатор и, если оно оказалось целым, приводит его к комплексному. Полученное значение записывается в память командой `CSTORE`. Реализация представлена в листинге 14.

Листинг 14 Метод `complexDeclaration`

```
1 // parser.cpp
2 void Parser::complexDeclaration()
3 {
4     if(!see(T_IDENTIFIER)) {
5         reportError("identifier expected after 'complex'.");
6         return;
7     }
8     string name = scanner_>getStringValue();
9     next();
10
11     if(variables_.find(name) != variables_.end()) {
12         reportError("variable '" + name + "' is already declared.");
13         return;
14     }
15     int addr = addVariable(name, VT_COMPLEX);
16
17     mustBe(T_ASSIGN);
18
19     VarType exprType = expression();
20     if(exprType == VT_ERROR) return;
21
22     if(exprType == VT_INT) {
23         promoteIntToComplex();
24     }
25     codegen_>emit(CSTORE, addr);
26 }
```

2.3.4 Разбор выражений со смешением типов

Методы `expression` и `term` изменены так, чтобы возвращать тип результата и корректно обрабатывать смешение целых и комплексных операндов. Компилятор однопроходный, поэтому для каждой операции рассматриваются четыре случая в зависимости от типов левого и правого операндов.

Если оба операнда целые, генерируются обычные инструкции ADD/SUB (или MULT/DIV в `term`), результат остаётся целым. Если оба операнда комплексные, генерируются комплексные инструкции CADD/CSUB (CMULT/CDIV). Если левый операнд комплексный, а правый целый, то правый (находящийся на вершине стека) приводится к комплексному, после чего выполняется комплексная операция. Если левый операнд целый, а правый комплексный, то в этом случае комплексный правый операнд временно сохраняется в служебные ячейки памяти, целый левый операнд приводится к комплексному, правый операнд восстанавливается, и выполняется комплексная операция. Реализация метода `expression` приведена в листинге 15. Изменены строки 4-5, 11-14, 17, 19-42.

Листинг 15 Метод `expression` с обработкой смешения типов

```

1  // parser.cpp
2  Parser::VarType Parser::expression()
3  {
4      VarType leftType = term();
5      if(leftType == VT_ERROR) return VT_ERROR;
6
7      while(see(T_ADDOP)) {
8          Arithmetic op = scanner_ ->getArithmeticValue();
9          next();
10
11         VarType rightType = term();
12         if(rightType == VT_ERROR) return VT_ERROR;
13
14         if(leftType == VT_INT && rightType == VT_INT) {
15             if(op == A_PLUS) codegen_ ->emit(ADD);
16             else codegen_ ->emit(SUB);
17             leftType = VT_INT;
18         }
19         else if(leftType == VT_INT && rightType == VT_COMPLEX) {
20             // левый int, потом правый complex сверху.
21             int tmpReR = TMP_SLOT_0;
22             int tmpReI = TMP_SLOT_1;
23             codegen_ ->emit(STORE, tmpReI); // im правого
24             codegen_ ->emit(STORE, tmpReR); // re правого
25             promoteIntToComplex();
26             codegen_ ->emit(LOAD, tmpReR);
27             codegen_ ->emit(LOAD, tmpReI);
28             if(op == A_PLUS) codegen_ ->emit(CADD);
29             else codegen_ ->emit(CSUB);
30             leftType = VT_COMPLEX;
31         }
32         else if(leftType == VT_COMPLEX && rightType == VT_INT) {
33             // левый complex, правый int сверху.
34             promoteIntToComplex();
35             if(op == A_PLUS) codegen_ ->emit(CADD);
36             else codegen_ ->emit(CSUB);
37             leftType = VT_COMPLEX;

```

```

38     }
39     else { // обе complex
40         if(op == A_PLUS) codegen_ ->emit(CADD);
41         else codegen_ ->emit(CSUB);
42         leftType = VT_COMPLEX;
43     }
44 }
45 return leftType;
46 }

```

Метод `term` построен аналогично, но для операций умножения и деления, генерируя инструкции `MULT/DIV` для целых операндов и `CMULT/CDIV` для комплексных, с такой же обработкой четырёх случаев смешения типов (листинг 16, изменены строки 4-5, 11-14, 17, 19-40).

Листинг 16 Метод `term` (умножение и деление)

```

1 // parser.cpp
2 Parser::VarType Parser::term()
3 {
4     VarType leftType = factor();
5     if(leftType == VT_ERROR) return VT_ERROR;
6
7     while(see(T_MULOP)) {
8         Arithmetic op = scanner_ ->getArithmeticValue();
9         next();
10
11         VarType rightType = factor();
12         if(rightType == VT_ERROR) return VT_ERROR;
13
14         if(leftType == VT_INT && rightType == VT_INT) {
15             if(op == A_MULTIPLY) codegen_ ->emit(MULT);
16             else codegen_ ->emit(DIV);
17             leftType = VT_INT;
18         }
19         else if(leftType == VT_INT && rightType == VT_COMPLEX) {
20             int tmpReR = TMP_SLOT_0;
21             int tmpReI = TMP_SLOT_1;
22             codegen_ ->emit(STORE, tmpReI);
23             codegen_ ->emit(STORE, tmpReR);
24             promoteIntToComplex();
25             codegen_ ->emit(LOAD, tmpReR);
26             codegen_ ->emit(LOAD, tmpReI);
27             if(op == A_MULTIPLY) codegen_ ->emit(CMULT);
28             else codegen_ ->emit(CDIV);
29             leftType = VT_COMPLEX;
30         }
31         else if(leftType == VT_COMPLEX && rightType == VT_INT) {
32             promoteIntToComplex();
33             if(op == A_MULTIPLY) codegen_ ->emit(CMULT);
34             else codegen_ ->emit(CDIV);
35             leftType = VT_COMPLEX;

```

```

36     }
37     else {
38         if (op == A_MULTIPLY) codegen_ ->emit(CMULT);
39         else codegen_ ->emit(CDIV);
40         leftType = VT_COMPLEX;
41     }
42 }
43 return leftType;
44 }

```

2.3.5 Литералы, унарные операции и функции

Метод `factor` расширен новыми конструкциями.

Унарные операции. Унарный минус разбирает следующий фактор и в зависимости от его типа генерирует `CINVERT` (для комплексного) или `INVERT` (для целого). Унарный плюс просто разбирает фактор без генерации команд (листинг 17, строки 5-22).

Числовые и мнимые литералы. При разборе числа после него проверяется наличие мнимой единицы. Если за числом следует `i`, фактор является чисто мнимым литералом и в стек помещается значение командой `CPUSHI`, а тип результата – комплексный. Иначе это обычный целый литерал, помещаемый командой `PUSH` (листинг 17, строки 28-32).

Мнимая единица. Если мнимая единица `i` встречается сама по себе, то в стек помещается значение $1 \cdot i$ командой `CPUSHI` с аргументом 1 (листинг 17, строки 39-43).

Идентификаторы. При разборе идентификатора по таблице переменных определяется его тип. Для комплексной переменной значение загружается командой `CLOAD`, для целой – командой `LOAD`; обращение к необъявленной переменной диагностируется как ошибка (листинг 17, строки 49-60).

Функции над комплексными числами. Для функций `real`, `imag`, `conj`, `abs` и `arg` введена вспомогательная лямбда-функция `parseComplexFunc`. Она разбирает аргумент в скобках, при необходимости приводит целый аргумент к комплексному и генерирует соответствующую инструкцию виртуальной машины. Реализация представлена в листинге 17, строки 78-94.

Листинг 17 Унарные операции в `factor`

```

1 // parser.cpp
2 Parser::VarType Parser::factor()
3 {
4     // Унарный минус
5     if (see(T_ADDOP) && scanner_ ->getArithmeticValue() == A_MINUS) {
6         next();
7         VarType t = factor();
8         if (t == VT_ERROR) return VT_ERROR;
9         if (t == VT_COMPLEX) {
10             codegen_ ->emit(CINVERT);

```



```

11     }
12     else {
13         codegen_ -> emit(INVERT);
14     }
15     return t;
16 }
17
18 // Унарный плюс
19 if(see(T_ADDOP) && scanner_ -> getArithmeticValue() == A_PLUS) {
20     next();
21     return factor();
22 }
23
24 // Числовой литерал
25 if(see(T_NUMBER)) {
26     int value = scanner_ -> getIntValue();
27     next();
28     if(see(T_IMAG_UNIT)) {
29         next();
30         // чисто мнимое value * i
31         codegen_ -> emit(CPUSHI, value);
32         return VT_COMPLEX;
33     }
34     codegen_ -> emit(PUSH, value);
35     return VT_INT;
36 }
37
38 // Мнимая единица
39 if(see(T_IMAG_UNIT)) {
40     next();
41     codegen_ -> emit(CPUSHI, 1);
42     return VT_COMPLEX;
43 }
44
45 // Идентификатор
46 if(see(T_IDENTIFIER)) {
47     string name = scanner_ -> getStringValue();
48     next();
49     VarInfo info;
50     if(!getVariable(name, info)) {
51         reportError("undeclared variable '" + name + "'.");
52         return VT_ERROR;
53     }
54     if(info.type == VT_COMPLEX) {
55         codegen_ -> emit(CLOAD, info.address);
56         return VT_COMPLEX;
57     }
58     else {
59         codegen_ -> emit(LOAD, info.address);
60         return VT_INT;
61     }
62 }

```

```

63
64 // Скобки
65 if(match(T_LPAREN)) {
66     VarType t = expression();
67     mustBe(T_RPAREN);
68     return t;
69 }
70
71 // READ
72 if(match(T_READ)) {
73     codegen_ ->emit(INPUT);
74     return VT_INT;
75 }
76
77 // Комплексные функции
78 auto parseComplexFunc = [&](Instruction op, VarType resultType)
79     ->VarType {
80     mustBe(T_LPAREN);
81     VarType t = expression();
82     mustBe(T_RPAREN);
83     if(t == VT_ERROR) return VT_ERROR;
84     if(t == VT_INT) {
85         promoteIntToComplex();
86     }
87     codegen_ ->emit(op);
88     return resultType;
89 };
90
91 if(match(T_REAL)) return parseComplexFunc(CREAL, VT_INT);
92 if(match(T_IMAG)) return parseComplexFunc(CIMAG, VT_COMPLEX);
93 if(match(T_CONJ)) return parseComplexFunc(CCONJ, VT_COMPLEX);
94 if(match(T_ABS)) return parseComplexFunc(CABS, VT_INT);
95 if(match(T_ARG)) return parseComplexFunc(CARG, VT_INT);
96
97 reportError("expression expected.");
98 return VT_ERROR;
99 }

```

2.3.6 Сравнение значений

Метод `relation` изменён так, чтобы учитывать типы сравниваемых выражений. Если хотя бы один из операндов комплексный, оба приводятся к комплексному типу, и допускаются только сравнения на равенство и неравенство; операции порядка диагностируются как ошибка. Для целых операндов сохраняется прежнее поведение с инструкцией `COMPARE` и полным набором операций сравнения. Реализация представлена в листинге 18. Добавлены строки 11-33.

Листинг 18 Метод `relation` с учётом типов

```

1 // parser.cpp
2 void Parser::relation()
3 {
4     VarType lt = expression();
5     if(!see(T_CMP)) {
6         reportError("comparison operator expected.");
7         return;
8     }
9     Cmp cmp = scanner_ ->getCmpValue();
10    next();
11    VarType rt = expression();
12    if(lt == VT_ERROR || rt == VT_ERROR) return;
13
14    if(lt == VT_COMPLEX || rt == VT_COMPLEX) {
15        if(rt == VT_INT) { promoteIntToComplex(); rt = VT_COMPLEX
16        , }
17        if(lt == VT_INT) {
18            codegen_ ->emit(STORE, TMP_SLOT_1);
19            codegen_ ->emit(STORE, TMP_SLOT_0);
20            promoteIntToComplex();
21            codegen_ ->emit(LOAD, TMP_SLOT_0);
22            codegen_ ->emit(LOAD, TMP_SLOT_1);
23            lt = VT_COMPLEX;
24        }
25        switch(cmp) {
26            case C_EQ: codegen_ ->emit(CCOMPARE, 0); break;
27            case C_NE: codegen_ ->emit(CCOMPARE, 1); break;
28            default:
29                reportError("only '=' and '!=' are allowed "
30                "for complex comparison.");
31                return;
32        }
33    }
34    else {
35        switch(cmp) {
36            case C_EQ: codegen_ ->emit(COMPARE, 0); break;
37            case C_NE: codegen_ ->emit(COMPARE, 1); break;
38            case C_LT: codegen_ ->emit(COMPARE, 2); break;
39            case C_GT: codegen_ ->emit(COMPARE, 3); break;
40            case C_LE: codegen_ ->emit(COMPARE, 4); break;
41            case C_GE: codegen_ ->emit(COMPARE, 5); break;
42        }
43    }
44 }

```

2.4 Новые инструкции генератора кода

В перечисление `Instruction` генератора кода (`codegen.h`) добавлены шестнадцать новых инструкций для работы с комплексными числами. Их перечень приведён в листинге 19.

Листинг 19 Добавленные инструкции в codegen.h

```

1 // codegen.h, enum Instruction
2 enum Instruction
3 {
4     NOP,
5     STOP,
6     LOAD,
7     STORE,
8     BLOAD,
9     BSTORE,
10    PUSH,
11    POP,
12    DUP,
13    INVERT,
14    ADD,
15    SUB,
16    MULT,
17    DIV,
18    COMPARE,
19    JUMP,
20    JUMP_YES,
21    JUMP_NO,
22    INPUT,
23    PRINT,
24    CPUSH,      // CPUSH n -> в стек кладётся комплексное (n, 0) с м
                   асштабом
25    CPUSHI,     // CPUSHI n -> в стек кладётся (0, n) с масштабом
26    CLOAD,      // CLOAD addr -> в стек кладётся пара (mem[addr],
                   mem[addr+1])
27    CSTORE,     // CSTORE addr -> с вершины (re, im) пишется в mem[
                   addr], mem[addr+1]
28    CADD,
29    CSUB,
30    CMULT,
31    CDIV,
32    CINVERT,
33    CCONJ,
34    CREAL,
35    CIMAG,
36    CABS,
37    CARG,
38    CPRINT,
39    CCOMPARE    // CCOMPARE n -> 0=EQ, 1=NE для двух комплексных
40 };

```

В метод `Command::print (codegen.cpp)` добавлены ветви для текстовой печати каждой новой инструкции в ассемблерном представлении. Инструкции с аргументом (`CPUSH`, `CPUSHI`, `CLOAD`, `CSTORE`, `CCOMPARE`) печатаются вместе со своим аргументом, остальные – без него (листинг 20).

Листинг 20 Печать новых инструкций в codegen.cpp

```

1 // codegen.cpp
2 void Command::print(int address, ostream& os)
3 {
4     os << address << ":\t";
5     switch(instruction_) {
6         case NOP:      os << "NOP";                break;
7         case STOP:     os << "STOP";                break;
8         case LOAD:     os << "LOAD\t"              << arg_; break;
9         case STORE:    os << "STORE\t"             << arg_; break;
10        case BLOAD:    os << "BLOAD\t"             << arg_; break;
11        case BSTORE:   os << "BSTORE\t"            << arg_; break;
12        case PUSH:     os << "PUSH\t"              << arg_; break;
13        case POP:      os << "POP";                 break;
14        case DUP:      os << "DUP";                 break;
15        case ADD:      os << "ADD";                 break;
16        case SUB:      os << "SUB";                 break;
17        case MULT:     os << "MULT";                break;
18        case DIV:      os << "DIV";                 break;
19        case INVERT:   os << "INVERT";              break;
20        case COMPARE:  os << "COMPARE\t"           << arg_; break;
21        case JUMP:     os << "JUMP\t"              << arg_; break;
22        case JUMP_YES: os << "JUMP_YES\t"          << arg_; break;
23        case JUMP_NO:  os << "JUMP_NO\t"           << arg_; break;
24        case INPUT:    os << "INPUT";               break;
25        case PRINT:    os << "PRINT";               break;
26        case CPUSH:    os << "CPUSH\t"              << arg_; break;
27        case CPUSHI:   os << "CPUSHI\t"            << arg_; break;
28        case CLOAD:    os << "CLOAD\t"             << arg_; break;
29        case CSTORE:   os << "CSTORE\t"            << arg_; break;
30        case CADD:     os << "CADD";                break;
31        case CSUB:     os << "CSUB";                break;
32        case CMULT:    os << "CMULT";               break;
33        case CDIV:     os << "CDIV";               break;
34        case CINVERT:  os << "CINVERT";             break;
35        case CCONJ:    os << "CCONJ";               break;
36        case CREAL:    os << "CREAL";               break;
37        case CIMAG:    os << "CIMAG";               break;
38        case CABS:     os << "CABS";                break;
39        case CARG:     os << "CARG";                break;
40        case CPRINT:   os << "CPRINT";              break;
41        case CCOMPARE: os << "CCOMPARE\t"          << arg_; break;
42    }
43    os << endl;
44 }

```

2.5 Доработка виртуальной машины

В перечисление операций `operation` (`vm.h`) добавлены те же шестнадцать инструкций, что и в генераторе кода. Перечень добавленных операций приведён в листинге 21.

```

1 // vm.h
2 typedef enum {
3     NOP = 0,           /* нет действия */
4     STOP,              /* остановка */
5     LOAD,              /* загрузка слова из памяти данных */
6     STORE,             /* запись слова в память данных */
7     BLOAD,             /* загрузка слова из памяти данных с учётом
                        смещения */
8     BSTORE,            /* запись слова в память данных с учётом
                        смещения */
9     PUSH,              /* проталкивание слова в стек */
10    POP,               /* выталкивание слова из стека */
11    DUP,              /* дублирование слова на вершине стека */
12    INVERT,           /* инверсия знака слова на вершине стека
                        */
13    ADD,              /* сложение */
14    SUB,              /* вычитание */
15    MULT,             /* умножение */
16    DIV,              /* целочисленное деление */
17    COMPARE,          /* сравнение */
18    JUMP,             /* безусловный переход */
19    JUMP_YES,          /* условный переход, если на вершине стек
                        а НЕ 0 */
20    JUMP_NO,           /* условный переход, если на вершине стек
                        а 0 */
21    INPUT,            /* чтение числа со стандартного устройства
                        а ввода */
22    PRINT,            /* печать числа на стандартное устройство
                        вывода */
23    CPUSH,             /* CPUSH re - проталкивает (re, 0) - цело
                        е в виде комплексного */
24    CPUSHI,           /* CPUSHI im - проталкивает (0, im) - чис
                        то мнимое */
25    CLOAD,            /* CLOAD addr - загружает в стек (mem[
                        addr], mem[addr+1]) */
26    CSTORE,           /* CSTORE addr - записывает с вершины (re
                        , im) в mem[addr], mem[addr+1] */
27    CADD,             /* сложение комплексных */
28    CSUB,             /* вычитание комплексных */
29    CMULT,            /* умножение комплексных */
30    CDIV,             /* деление комплексных */
31    CINVERT,          /* унарный минус для комплексного */
32    CCONJ,            /* сопряжение */
33    CREAL,            /* выделение действительной части */
34    CIMAG,            /* выделение мнимой части */
35    CABS,             /* модуль */
36    CARG,             /* аргумент */
37    CPRINT,           /* печать комплексного числа */
38    CCOMPARE          /* сравнение двух комплексных */
39 } operation;

```

Для распознавания новых команд при разборе ассемблерного текста в таблицу `opcodes_table` (`vm.c`) добавлены записи с названием каждой команды и признаком наличия аргумента (листинг 22).

Листинг 22 Добавленные записи в таблицу `opcodes_table`

```

1 // vm.c
2 opcode_info opcodes_table[] = {
3     {"NOP", 0},
4     {"STOP", 0},
5     {"LOAD", 1},
6     {"STORE", 1},
7     {"BLOAD", 1},
8     {"BSTORE", 1},
9     {"PUSH", 1},
10    {"POP", 0},
11    {"DUP", 0},
12    {"INVERT", 0},
13    {"ADD", 0},
14    {"SUB", 0},
15    {"MULT", 0},
16    {"DIV", 0},
17    {"COMPARE", 1},
18    {"JUMP", 1},
19    {"JUMP_YES", 1},
20    {"JUMP_NO", 1},
21    {"INPUT", 0},
22    {"PRINT", 0},
23    {"CPUSH", 1},
24    {"CPUSHI", 1},
25    {"CLOAD", 1},
26    {"CSTORE", 1},
27    {"CADD", 0},
28    {"CSUB", 0},
29    {"CMULT", 0},
30    {"CDIV", 0},
31    {"CINVERT", 0},
32    {"CCONJ", 0},
33    {"CREAL", 0},
34    {"CIMAG", 0},
35    {"CABS", 0},
36    {"CARG", 0},
37    {"CPRINT", 0},
38    {"CCOMPARE", 1}
39 };

```

В `vm.c` добавлены три вспомогательные функции для работы с комплексными числами на стеке.

Функция `vm_pop_complex`:

Вход: указатели `int *re` и `int *im`, по которым будет записан результат; неявно — на вершине стека виртуальной машины лежат два значения.

Выход: нет возвращаемого значения; по `*im` записана мнимая часть, по

***re** — действительная; оба удалены со стека.

Функция `vm_push_complex`:

Вход: `int re` — действительная часть, `int im` — мнимая часть.

Выход: нет возвращаемого значения; на стек помещены `re` и `im`.

Реализация обеих функций представлена в листинге 23.

Листинг 23 Снятие и помещение комплексного числа на стек

```
1 // vm.c
2 static void vm_pop_complex(int *re, int *im)
3 {
4     *im = vm_pop();
5     *re = vm_pop();
6 }
7
8 static void vm_push_complex(int re, int im)
9 {
10    vm_push(re);
11    vm_push(im);
12 }
```

Функция `vm_write_complex`:

Вход: `int re_s` — действительная часть в формате с фиксированной точкой; `int im_s` — мнимая часть в том же формате.

Выход: нет возвращаемого значения; в поток `stdout` выведена строка с числом в формате `a+bi`, `a-bi`, `a` (если `b = 0`) или `bi` (если `a = 0`).

Реализация представлена в листинге 24.

Листинг 24 Печать комплексного числа

```
1 // vm.c
2 static void vm_write_complex(int re_s, int im_s)
3 {
4     double re = (double)re_s / (double)VM_SCALE;
5     double im = (double)im_s / (double)VM_SCALE;
6
7     char re_buf[64], im_buf[64];
8
9     if(im_s == 0) {
10        format_number(re, re_buf, sizeof(re_buf));
11        fprintf(stdout, "%s\n", re_buf);
12    }
13    else if(re_s == 0) {
14        format_number(im, im_buf, sizeof(im_buf));
15        fprintf(stdout, "%si\n", im_buf);
16    }
17    else if(im > 0) {
18        format_number(re, re_buf, sizeof(re_buf));
19        format_number(im, im_buf, sizeof(im_buf));
20        fprintf(stdout, "%s+%si\n", re_buf, im_buf);
21    }
22    else {
```



```

23     format_number(re, re_buf, sizeof(re_buf));
24     format_number(im, im_buf, sizeof(im_buf));
25     fprintf(stdout, "%s%s\n", re_buf, im_buf);
26 }
27 fflush(stdout);
28 }

```

В основной цикл выполнения команд `vm_run_command` (`vm.c`) добавлена обработка каждой новой инструкции.

Инструкции `CPUSH` и `CPUSHI` помещают на стек комплексное число, полученное из целого аргумента: `CPUSH` формирует $(arg \cdot SCALE, 0)$, `CPUSHI` – $(0, arg \cdot SCALE)$. Инструкции `CLOAD` и `CSTORE` обмениваются парой слов с памятью данных по заданному адресу (листинг 25).

Листинг 25 Загрузка и сохранение комплексных значений

```

1 // vm.c, vm_run_command
2 case CPUSH:
3     vm_push_complex((int)arg * VM_SCALE, 0);
4     break;
5 case CPUSHI:
6     vm_push_complex(0, (int)arg * VM_SCALE);
7     break;
8 case CLOAD: {
9     int re = vm_load(arg);
10    int im = vm_load(arg + 1);
11    vm_push_complex(re, im);
12    break;
13 }
14 case CSTORE: {
15     int re, im;
16     vm_pop_complex(&re, &im);
17     vm_store(arg, re);
18     vm_store(arg + 1, im);
19     break;
20 }

```

Инструкции `CADD` и `CSUB` реализуют покомпонентное сложение и вычитание (листинг 26).

Листинг 26 Сложение и вычитание комплексных чисел

```

1 // vm.c, vm_run_command
2 case CADD: {
3     int a, b, c, d;
4     vm_pop_complex(&c, &d);
5     vm_pop_complex(&a, &b);
6     vm_push_complex(a + c, b + d);
7     break;
8 }
9 case CSUB: {
10    int a, b, c, d;
11    vm_pop_complex(&c, &d);

```

```

12     vm_pop_complex(&a, &b);
13     vm_push_complex(a - c, b - d);
14     break;
15 }

```

Инструкции CMULT и CDIV реализуют умножение и деление по формулам комплексной арифметики. Поскольку числа хранятся с фиксированной точкой, в промежуточных вычислениях используется 64-разрядный тип `long long` и деление на масштаб для компенсации удвоения множителя; при делении проверяется обращение знаменателя в нуль (листинг 27).

Листинг 27 Умножение и деление комплексных чисел

```

1  // vm.c, vm_run_command
2  case CMULT: {
3      long long a, b, c, d, re, im;
4      int ai, bi, ci, di;
5      vm_pop_complex(&ci, &di);
6      vm_pop_complex(&ai, &bi);
7      a = ai; b = bi; c = ci; d = di;
8      re = (a * c - b * d) / VM_SCALE;
9      im = (a * d + b * c) / VM_SCALE;
10     vm_push_complex((int)re, (int)im);
11     break;
12 }
13 case CDIV: {
14     long long a, b, c, d, denom, re, im;
15     int ai, bi, ci, di;
16     vm_pop_complex(&ci, &di);
17     vm_pop_complex(&ai, &bi);
18     a = ai; b = bi; c = ci; d = di;
19     denom = (c * c + d * d) / VM_SCALE;
20     if(denom == 0) { vm_error(DIVISION_BY_ZERO); break; }
21     re = ((a * c + b * d) / VM_SCALE) * VM_SCALE / denom;
22     im = ((b * c - a * d) / VM_SCALE) * VM_SCALE / denom;
23     vm_push_complex((int)re, (int)im);
24     break;
25 }

```

Инструкции CINVERT и CCONJ реализуют унарный минус и комплексное сопряжение. Инструкция CREAL выделяет действительную часть и возвращает её как целое значение. Инструкция CIMAG выделяет мнимую часть и возвращает комплексную пару с нулевой вещественной частью.

Листинг 28 Унарный минус, сопряжение и выделение частей

```

1  // vm.c, vm_run_command
2  case CINVERT: {
3      int re, im; vm_pop_complex(&re, &im);
4      vm_push_complex(-re, -im); break;
5  }
6  case CCONJ: {
7      int re, im; vm_pop_complex(&re, &im);

```

```

8     vm_push_complex(re, -im); break;
9 }
10 case CREAL: {
11     int re, im;
12     vm_pop_complex(&re, &im);
13     (void)im;
14     vm_push(re / VM_SCALE);
15     break;
16 }
17 case CIMAG: {
18     int re, im;
19     vm_pop_complex(&re, &im);
20     (void)re;
21     vm_push_complex(0, im);
22     break;
23 }

```

Инструкции CABS и CARG вычисляют модуль и аргумент комплексного числа. Результат в обоих случаях возвращается как целое число (листинг 29).

Листинг 29 Модуль и аргумент комплексного числа

```

1 // vm.c, vm_run_command
2 case CABS: {
3     int re, im;
4     double r;
5     vm_pop_complex(&re, &im);
6     r = sqrt((double)re * (double)re + (double)im * (double)im);
7     vm_push((int)(r / VM_SCALE));
8     break;
9 }
10
11 case CARG: {
12     int re, im;
13     double a;
14     vm_pop_complex(&re, &im);
15     a = atan2((double)im, (double)re);
16     vm_push((int)(a * VM_SCALE));
17     break;
18 }

```

Инструкция CPRINT снимает со стека комплексное число и печатает его при помощи функции `vm_write_complex`. Инструкция CCOMPARE снимает два комплексных числа, сравнивает их на равенство поэлементно и в зависимости от аргумента (EQ или NE) помещает на стек целочисленный признак истинности (листинг 30).

Листинг 30 Печать и сравнение комплексных чисел

```

1 // vm.c, vm_run_command
2 case CPRINT: {
3     int re, im;
4     vm_pop_complex(&re, &im);

```

```

5     vm_write_complex(re, im);
6     break;
7 }
8 case CCOMPARE: {
9     int a, b, c, d, eq;
10    vm_pop_complex(&c, &d);
11    vm_pop_complex(&a, &b);
12    eq = (a == c && b == d) ? 1 : 0;
13    if(arg == EQ)        vm_push(eq);
14    else if(arg == NE)   vm_push(eq ? 0 : 1);
15    else                 vm_error(BAD_RELATION);
16    break;
17 }

```

3 Результаты работы программы

Программа №1: объявление и печать

BEGIN

```
COMPLEX z := 2 + 3 i;  
COMPLEX w := 4 - 5 i;  
WRITE(z);  
WRITE(w)
```

END

Код компилятора

Результат вывода

0: PUSH 2	2+3i
1: CPUSHI 3	
2: STORE 65531	4-5i
3: STORE 65530	
4: PUSH 1000	
5: MULT	
6: PUSH 0	
7: LOAD 65530	
8: LOAD 65531	
9: CADD	
10: CSTORE 0	
11: PUSH 4	
12: CPUSHI 5	
13: STORE 65531	
14: STORE 65530	
15: PUSH 1000	
16: MULT	
17: PUSH 0	
18: LOAD 65530	
19: LOAD 65531	
20: CSUB	
21: CSTORE 2	
22: CLOAD 0	
23: CPRINT	
24: CLOAD 2	
25: CPRINT	
26: STOP	

Программа №2: арифметические операции

BEGIN

```
COMPLEX z := 2 + 3 i;  
COMPLEX w := 1 - 4 i;  
WRITE(z + w);  
WRITE(z - w);  
WRITE(z * w);  
WRITE(z / w)
```

END

Код компилятора

```
0: PUSH 2
1: CPUSHI 3
2: STORE 65531
3: STORE 65530
4: PUSH 1000
5: MULT
6: PUSH 0
7: LOAD 65530
8: LOAD 65531
9: CADD
10: CSTORE 0
11: PUSH 1
12: CPUSHI 4
13: STORE 65531
14: STORE 65530
15: PUSH 1000
16: MULT
17: PUSH 0
18: LOAD 65530
19: LOAD 65531
20: CSUB
21: CSTORE 2
22: CLOAD 0
23: CLOAD 2
24: CADD
25: CPRINT
26: CLOAD 0
27: CLOAD 2
28: CSUB
29: CPRINT
30: CLOAD 0
31: CLOAD 2
32: CMULT
33: CPRINT
34: CLOAD 0
35: CLOAD 2
36: CDIV
37: CPRINT
38: STOP
```

Результат вывода

```
3-1i
1+7i
14-5i
-0.588+0.647i
```

Программа №3: функции для комплексных чисел

BEGIN

```
COMPLEX z := 3 + 4 i;
WRITE(REAL(z));
WRITE(IMAG(z));
WRITE(CONJ(z));
WRITE(ABS(z));
WRITE(ARG(z))
```

END

Код компилятора

```
0: PUSH 3
1: CPUSHI 4
2: STORE 65531
3: STORE 65530
4: PUSH 1000
5: MULT
6: PUSH 0
7: LOAD 65530
8: LOAD 65531
9: CADD
10: CSTORE 0
11: CLOAD 0
12: CREAL
13: CPRINT
14: CLOAD 0
15: CIMAG
16: CPRINT
17: CLOAD 0
18: CCONJ
19: CPRINT
20: CLOAD 0
21: CABS
22: CPRINT
23: CLOAD 0
24: CARG
25: CPRINT
26: STOP
```

Результат вывода

```
3
4
3-4i
5
0.927
```

Программа №4: смешивание целых и комплексных чисел

BEGIN

```
COMPLEX z := 5 + 2i;
WRITE(z + 3);
WRITE(10 + z);
WRITE(z * 2);
WRITE(-z);
COMPLEX w := 7;
WRITE(w)
```

END

Код компилятора

```
0: PUSH 5
1: CPUSHI 2
2: STORE 65531
3: STORE 65530
4: PUSH 1000
5: MULT
6: PUSH 0
7: LOAD 65530
8: LOAD 65531
9: CADD
```

Результат вывода

```
8+2i
15+2i
10+4i
-5-2i
7
```

```

10: CSTORE 0
11: CLOAD 0
12: PUSH 3
13: PUSH 1000
14: MULT
15: PUSH 0
16: CADD
17: CPRINT
18: PUSH 10
19: CLOAD 0
20: STORE 65531
21: STORE 65530
22: PUSH 1000
23: MULT
24: PUSH 0
25: LOAD 65530
26: LOAD 65531
27: CADD
28: CPRINT
29: CLOAD 0
30: PUSH 2
31: PUSH 1000
32: MULT
33: PUSH 0
34: CMULT
35: CPRINT
36: CLOAD 0
37: CINVERT
38: CPRINT
39: PUSH 7
40: PUSH 1000
41: MULT
42: PUSH 0
43: CSTORE 2
44: CLOAD 2
45: CPRINT
46: STOP

```

Программа №5: сравнение комплексных чисел

```

BEGIN
  COMPLEX z := 1 + 2 i;
  COMPLEX w := 1 + 2 i;
  IF z = w THEN
    WRITE(1)
  ELSE
    WRITE(0)
  FI
END

```

Код компилятора

```

0: PUSH 1
1: CPUSHI 2
2: STORE 65531
3: STORE 65530
4: PUSH 1000
5: MULT
6: PUSH 0

```

Результат вывода

1


```

7: LOAD 65530
8: LOAD 65531
9: CADD
10: CSTORE 0
11: PUSH 1
12: CPUSHI 2
13: STORE 65531
14: STORE 65530
15: PUSH 1000
16: MULT
17: PUSH 0
18: LOAD 65530
19: LOAD 65531
20: CADD
21: CSTORE 2
22: CLOAD 0
23: CLOAD 2
24: CCOMPARE 0
25: JUMP_NO 29
26: PUSH 1
27: PRINT
28: JUMP 31
29: PUSH 0
30: PRINT
31: STOP

```

Программа №6: возведение комплексного числа в степень

```

/* (1+i)^8 = 16 */
BEGIN
  COMPLEX z := 1 + 1 i;
  COMPLEX p := 1;
  k := 0;
  WHILE k < 8 do
    p := p * z;
    k := k + 1
  OD;
  WRITE(p)
END

```

Код компилятора

```

0: PUSH 1
1: CPUSHI 1
2: STORE 65531
3: STORE 65530
4: PUSH 1000
5: MULT
6: PUSH 0
7: LOAD 65530
8: LOAD 65531
9: CADD
10: CSTORE 0
11: PUSH 1
12: PUSH 1000
13: MULT
14: PUSH 0
15: CSTORE 2

```

Результат вывода

16

```

16: PUSH 0
17: STORE 4
18: LOAD 4
19: PUSH 8
20: COMPARE 2
21: JUMP_NO 31
22: CLOAD 2
23: CLOAD 0
24: CMULT
25: CSTORE 2
26: LOAD 4
27: PUSH 1
28: ADD
29: STORE 4
30: JUMP 18
31: CLOAD 2
32: CPRINT
33: STOP

```

Программа №7 (с ошибками)

```

BEGIN
  INT z := 1 + 1 i;
  COMPLEX p := 1;
  IF p > 8 then
    p := p * z;
    k := k + 1
  FI;
  WRITE(p)
END

```

Код компилятора и результат

```

Line 3: identifier found while ':= ' expected.
Line 3: cannot assign complex value to int variable 'int'.
Line 5: only '=' and '!=' are allowed for complex comparison.
Line 6: undeclared variable 'z'.

```

Заключение

В ходе выполнения лабораторной работы компилятор языка MiLan, относящегося к классу контекстно-свободных, был расширен поддержкой типа данных `complex` и операций над комплексными числами.

Лексический и синтаксический анализаторы были расширены путем добавления типа данных `complex`, который поддерживает четыре арифметические операции сложения, вычитания, умножения и деления, функции `real`, `imag`, `conj`, `abs`, `arg`, сравнение на равенство и неравенство. Также реализовано автоматическое приведение типа `int` к `complex`, операция унарного минуса и диагностика ошибок типизации при попытке присвоить комплексное значение целочисленной переменной.

После внесенных изменений синтаксический анализатор так же относится в типу LL(1) и использует метод рекурсивного спуска.

Достоинства реализации.

1. Все изменения реализованы в новых ветвях `switch` и дополнительных методах парсера; «целочисленная» часть кода не затронута, поэтому ранее написанные программы на MiLan транслируются и исполняются без изменений.
2. Присваивание значения типа `complex` целочисленной переменной, а также применение операторов порядка к комплексным числам обнаруживаются на этапе компиляции с указанием номера строки.
3. Комплексные числа представлены парой масштабированных целых чисел (`SCALE = 1000`), что позволило сохранить стековую VM полностью целочисленной и избежать введения типа `double`.

Недостатки реализации.

1. Масштаб `SCALE = 1000` обеспечивает лишь три знака после запятой. При многократном умножении или делении накапливается погрешность округления, которая может быть заметна уже на нескольких итерациях.
2. Оператор `/` выполняет целочисленное деление для операндов типа `int` и математически точное деление для операндов типа `complex`: выражение `7 / 2` даёт 3, тогда как `(7+0i) / (2+0i)` даёт 3.5. Это нарушает принцип единообразия поведения оператора.

Масштабирование программы.

1. **Добавление новых функций.** Добавление таких функций, как `sqrt`, `exp`, `log`, `sin`, `cos` и т.д.

2. **Повышение точности.** Для увеличения точности достаточно изменить константу `VM_SCALE` одновременно в `parser.cpp` и `vm.h`.
3. **Устранение несогласованности деления.** Чтобы оператор `/` вёл себя единообразно для всех типов, достаточно изменить реализацию опкода `DIV` по образцу `CDIV`: умножать делимое на `SCALE` перед делением. Для целочисленного деления можно ввести отдельный оператор `div`.

Список литературы

- [1] Секция «Телематика» ВШТИИ. – [\[Электронный ресурс\]](#) (дата обращения: 01.05.2026).
- [2] **Карпов Ю. Г.** Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005 – 272 с.